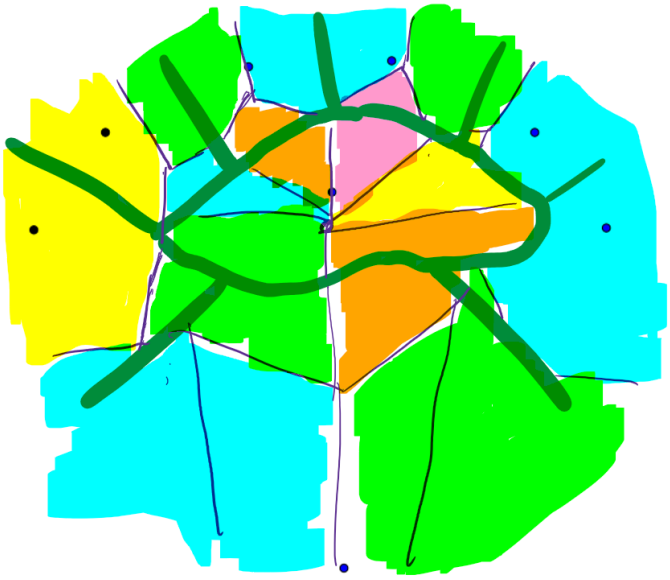
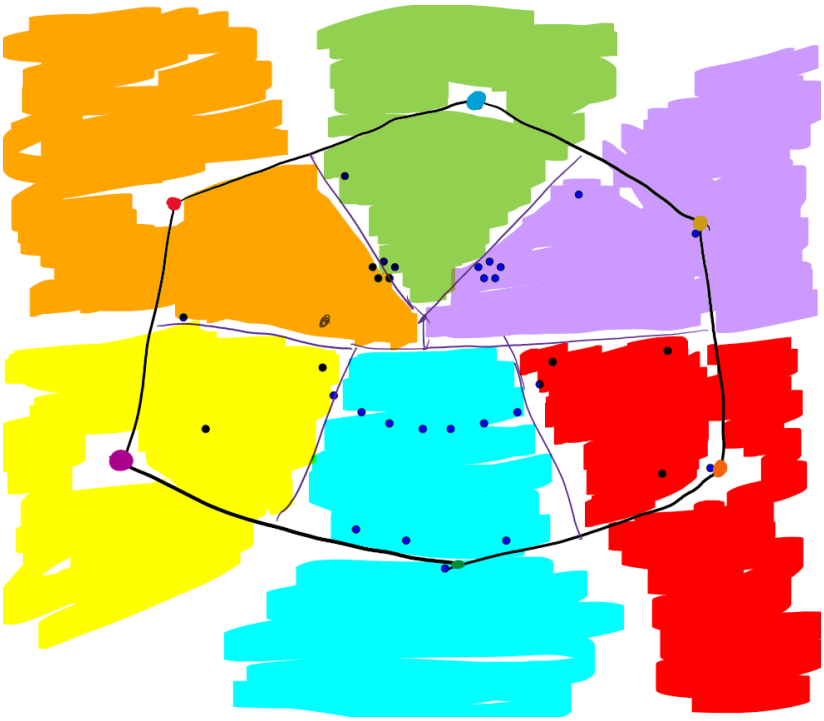
Bishoy Pramanik

Problem 1





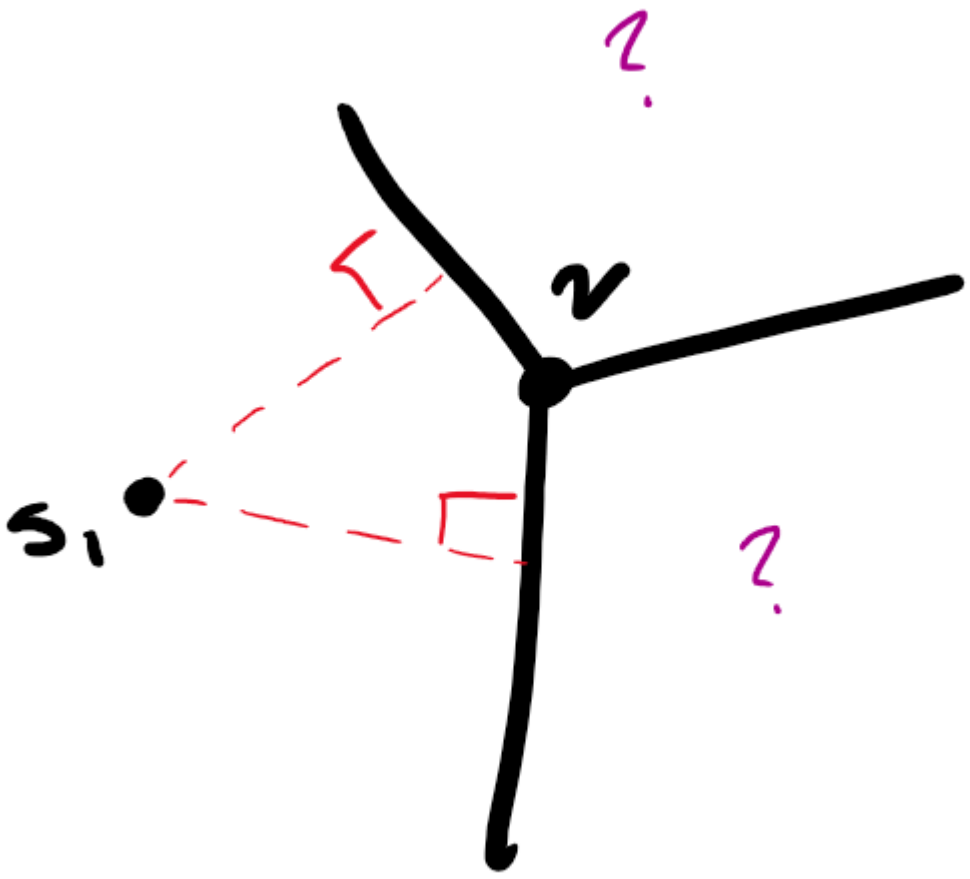
Problem 2

Please see the attached p2_submission.pdf for the solution and implementation.

Problem 3

Hint gotten from: https://cs.tulane.edu/~carola/teaching/cmps3130-6130/spring20/homework/hw-grad-6.pdf

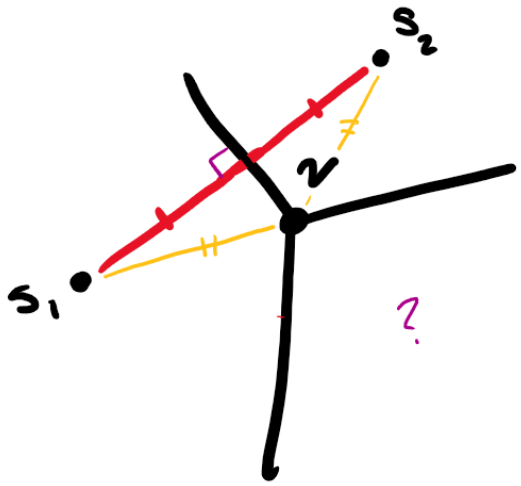
Consider a Voronoi diagram of three points, where we only know the location of one site.



We know that the Voronoi edges for the site are perpendicular bisectors to the lines created between the known site and each of the two adjacent sites. This means we can reflect the location of the known site of the respective Voronoi edge to find the other two unknown sites. If this mirrored flip did not align perfectly for all three points, then we would know definitively that this is not a Voronoi diagram, because the requirement of the perpendicular bisector would be violated. This also means, given the location of one site, we could cascade to finding all other points on the Voronoi diagram.

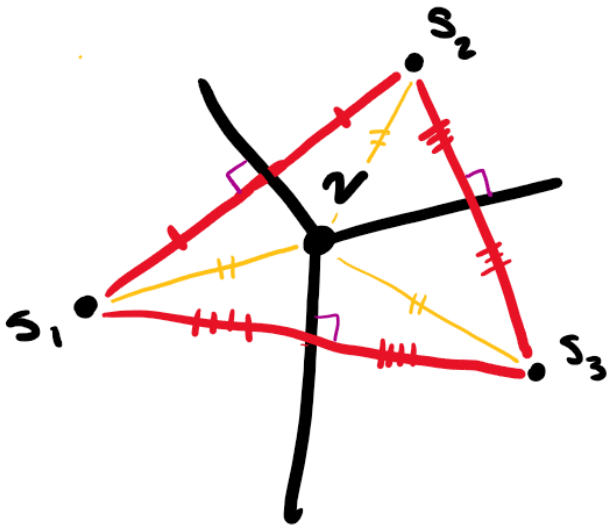
Additionally, the Voronoi vertex is the circumcenter for the circle created from all three sites. This means that the distance from each site to the Voronoi vertex is equal since they all lie on the circumference of the circle.

Because we can construct the constraint that sites must be equidistant from their shared Voronoi vertices and must be equidistant to their shared Voronoi edges, we can use the system of equations generated to solve for the first two potential sites.



We can pick any Voronoi region to start and place the first site anywhere within the region. We can then look at an adjacent Voronoi region and its corresponding Voronoi edge and vertex. The two linear lines give a system of equations that can be solved to give the other potential site location. By reflecting a third time, we get the third shared site.

Crucially, we can represent all other locations as a function of the first site location, and any Voronoi edges. For example: S_1: (x_1, y_1), S_2: (reflect(S_1, Edge(S_1, S_2)). All calculations would be built as such, and would propagate back to the location of S_1.



Notably, this does not give the exact site locations, since this does not fully define the site locations, but it fixes their locations relative to each other. The define site locations cannot rotate, translate, or

reflect. This means if we consider the distances to any other site locations, they can be calculated 'relative' to the distances used in the first triangle we constructed. We can now propagate to all other Voronoi regions through adjacent regions to what we've constructed (could be done through BFS or DFS) and propagate back to the first site location. If we do not end up at the exact same spot after propagating through all other regions, then this is not a Voronoi diagram. If we do, then it is potentially a Voronoi diagram. To verify that we did not end up at the beginning by chance, we can simply build a Voronoi diagram on the assumed sites and if the diagrams match, then it is a valid Voronoi diagram.

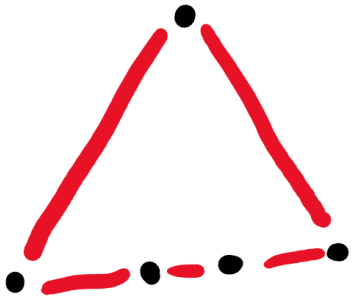
This algorithm searches through all n site locations and then the starting location one last time, meaning no revisited sites (except one). At each location, a reflection is calculated to assume the position of the next site location, which is a constant time operation. This means visiting all locations and propagating locations in total takes $O(n)$ time.

For verification, constructing a Voronoi diagram takes $O(n \log n)$ time. In total, my proposed algorithm takes $O(n \log n)$.

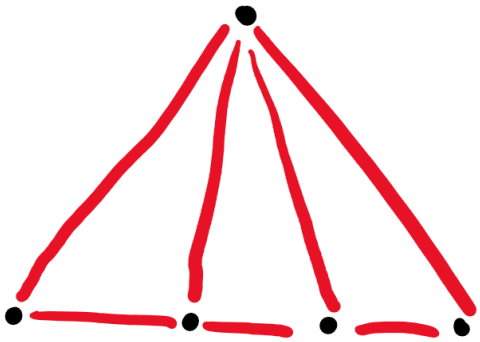
Problem 4

a) Describe a set of points for triangulation to maintain a degree of n-1

One of the properties of triangulation is that the triangulation must be made from the convex hull of the set of points. Consider a set of points where all lie on the convex hull, but all but one point are collinear, forming a triangle with a bottom edge where $n - 1$ points lie. The point that is not collinear is the **apex**.

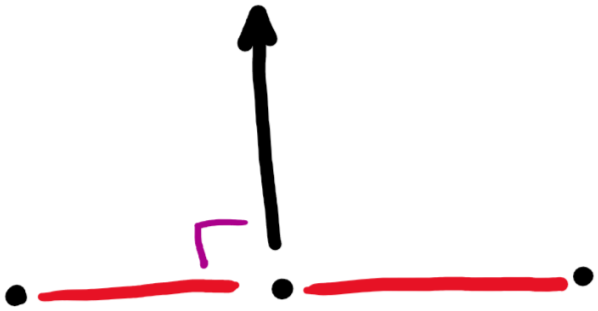


Because the points on the bottom edge are collinear, they can *only* connect to the apex to form a triangle. Because all $n - 1$ points must be connected to the apex point, the degree of the apex point is $n - 1$.



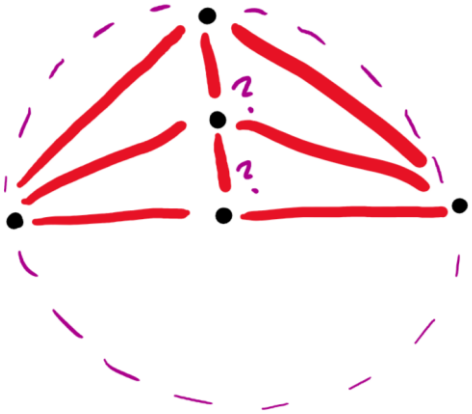
b) Worst-Case Scenario for a Randomized Incremental Algorithm

The worst-case scenario spurs from the empty-circle condition that must be enforced by the randomized insertion algorithm. Let's consider three evenly distributed, collinear points. These points cannot create a triangle with each other, so a Delaunay triangle does not apply here. But for any $n > 3$, we will insert a point along the perpendicular ray that comes out of the center point.



The first insertion causes all the points on the bottom edge to connect to the apex created.

But the subsequent insertion of point k along the ray creates a circumcircle with the end points of the bottom edge that contain all points within the point set.

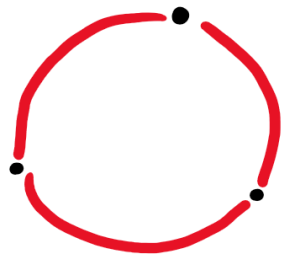


Because of this large, spanning circumcircle, all previously inserted points along the ray must be visited to verify that point $k-1$ could not create a better triangle. The triangle already exists, so no new triangle is made, but the edge from the end point to point k is flipped to be from the end point to point $k-1$. This new triangle has a circumcircle that contains point $k-2$, and the process continues until all the points along the ray are checked. This causes $O(k)$ work on every k insertion, leading to a $O(\frac{n(n-1)}{2})$ run-time, or $O(n^2)$.

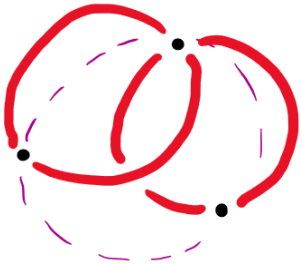
Problem 5

Gabriel Graphs are a subset of Delaunay Triangulations

A Delaunay triangulation edge, (a, b), exists if no point exists **inside a circle** passing through a and b.



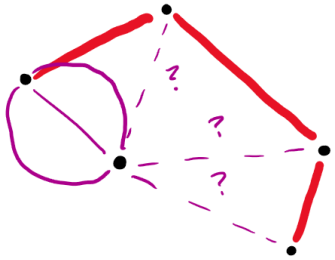
A Gabriel graph edge exists if no point exists **inside the specific circle** created by a and b.



A Gabriel edge must be a Delaunay edge because the specific circle has no points inside of it, therefore satisfying the condition of a Delaunay edge. This means all edges in the Gabriel graph satisfy the constraints of being Delaunay Triangulations.

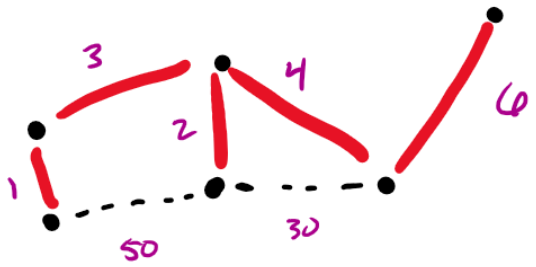
Relative Neighborhood Graphs are a subset of Gabriel Graphs

Relative neighborhood graphs connect points by assessing which point they are closest on the already connected graph.



Because the closest point on the existing graph is connected to the new point, this must be a Gabriel edge. This is because the closest point on the graph cannot have any other points on the connected graph closer to the point. If there was a closer point, then a and b would not be connected.

Euclidean Minimum Spanning Tree is a subset of a Relative Neighborhood Graph



Let's say there is an edge ab that is on the EMST that is not on the RNG. If this is true, there are two points connected on the EMST that are closer by distance than any other points on the connected graph. This would contradict the definition of the edges on the RNG, meaning the EMST must be comprised of edges that already exist on the RNG.